

基于公开基准与任务链需求的大语言模型综合评价及工程选型

摘要

公开 Benchmark 在任务构念、评分尺度和评测协议上口径不一，直接相加或平均难以同时保证可比性与可解释性。本文冻结 2026 年 8 月 17 日 Artificial Analysis 统一测评数据，以“模型版本—推理档位—工具配置—评测协议”的组合为评价单元，得到 27 个质量观测完整配置和 20 个工程字段可比配置，建立从公开测评到工程选型的统一评价链 $X \rightarrow Z \rightarrow C \rightarrow S^G \rightarrow (S_{is}, I_{is}) \rightarrow D_s(B)$ ，分别回答总体评价、场景适用与工程选型三个递进问题。

针对问题一，先按任务构念归并冗余指标，再作正向化与分位数标准化，将八项核心 Benchmark 组织为科学与复杂推理、知识正确性、知识边界校准、长文本理解、代码与科学计算、软件工程与工具执行、指令交互与任务完成七项评价维度；在缺乏可靠偏好先验时，用最大熵原理确定无偏好权重。求得通用评价第 1 名为 Kimi K3 (max)，得分 0.824176，其长文本、科学代码与工具执行维度领先，知识边界校准相对较弱，官方资料与独立测评在优势短板位置上相互印证。

针对问题二，将每类场景拆分为五项可辨认任务，以事前编码刻画任务对各维度的依赖强度并形成场景权重，同时引入相对短板约束 ($\theta = 0.25$) 防止非核心优势补偿核心短板。三类场景综合得分首名依次为 Kimi K3 (科研长文本 0.777972)、Grok 4.6 (日常对话 0.792308)、Kimi K3 (代码开发 0.838612)；跨场景贡献分解表明，同一模型在不同场景排名变化主要源于维度权重与自身能力结构的匹配。

针对问题三，将场景得分、token 直接费用与统一协议时延代理分开建模，以三目标 Pareto 前沿和预算约束给出分段选型路径。在中等负载、冻结日价格和既定时延边界下，科研路径为 MiniMax-M3→Grok 4.6→Kimi K3，日常为 Muse Glimmer→Gemini 3.5 Flash-Lite→GLM-5.2，代码为 MiniMax-M3→GLM-5.2→Grok 4.6→Kimi K3，预算增加时推荐由低费用方案转向高场景得分方案。对私有化部署路径，用公开硬件参数把能耗接口实例化，估算表明电费占比极低、硬件固定成本才是主体，自建与采购的盈亏平衡约为日调用一万次。

本文的模型优势有三：“构念审查 + 相关性预警”区分统计冗余与实质冗余，避免指标重复计量；“任务链事前编码”使场景评价复用同一评价向量、无需重测 Benchmark；参数化组织全部情景参数，使模型可由实际机构替换任务规模、价格与服务要求后重新求解。结论限于冻结样本与本文设定，不外推为普适价格规律或绝对能力次序。

关键词：大语言模型评价；公开 Benchmark；任务链；场景决策；Pareto 前沿；成本约束

目录

1	问题重述与分析	3
1.1	问题重述	3
1.2	统一数学对象	3
1.3	总体技术路线	4
2	数据、假设与符号	4
2.1	评价单元与数据来源	4
2.2	数据准入与假设	5
2.3	主要符号	6
3	问题一：核心评价维度构建与通用评价	6
3.1	问题解析	6
3.2	方法选择	6
3.3	模型建立：七项核心评价维度	8
3.4	模型求解：通用综合评价	8
3.5	结果分析：通用排名与 Kimi K3	9
3.6	本问小结	10
4	问题二：任务链驱动的场景评价	10
4.1	问题解析	11
4.2	方法选择：任务链与事前编码	11
4.3	模型建立：场景权重	11
4.4	模型求解：场景得分、相对短板约束与贡献分解	12
4.5	结果分析：三场景排名	12
4.6	本问小结	14
5	问题三：性能-成本权衡与工程选型	14
5.1	问题解析	14
5.2	方法选择：质量、费用与时延分开决策	14
5.3	模型建立：工作负载与直接费用	14
5.4	模型建立：Pareto 与预算选择	15
5.5	模型求解与结果分析：中等负载预算路径	15
5.6	结果分析：成本-质量关系与工作负载敏感性	17
5.7	本问小结	18
6	模型检验、敏感性与稳健性分析	19
6.1	参数敏感性	19
6.2	模型结构稳健性	19
6.3	工程决策稳健性	19
6.4	误差来源与适用边界	19
6.5	模型局限	20
7	结论与建议	20
7.1	问题一结论	20
7.2	问题二结论	20
7.3	问题三结论	20
7.4	使用建议	20
	参考文献	22
A	支撑材料与复现清单	22
B	AI 工具使用说明	23
C	核心代码节选	23

1 问题重述与分析

1.1 问题重述

赛题要求依据公开测评数据，对主流大语言模型进行**综合评价**，并回答三个递进问题。问题一关注模型总体表现：面对口径不一的公开 **Benchmark**，如何建立可比且可解释的通用评价。问题二关注使用条件：同一配置在科研长文本分析、大众日常通用对话和计算机代码开发中的适用性是否相同，场景需求如何改变评价维度的重要性。问题三关注部署决策：在场景得分确定后，如何加入价格、响应时延和计算资源约束，给出预算可执行的模型选择。

三问并非三个相互独立的排名任务。问题一将 **Benchmark** 原始观测组织为统一评价向量；问题二以该向量为固定输入，只改变任务需求；问题三再将场景得分与工程变量结合，形成带约束的决策。因此全文将题目抽象为

$$\boxed{\text{公开测评} \rightarrow \text{核心评价维度} \rightarrow \text{通用评价} \rightarrow \text{场景评价} \rightarrow \text{工程选型}}. \quad (1)$$

把这条评价链落到实处，难点集中在四件事上。**Benchmark** 多且彼此**信息重叠**：两个测评可能给出相近排序，任务构念却并不相同，相关性只能提出冗余预警，不能单独决定删除。各观测口径又不一致，必须先立准入规则、统一方向和尺度，缺失值不能当作低分。同一项能力在不同场景里的价值还不一样，长文本、日常对话和代码开发对应不同的任务链。最后，质量与部署代价天然冲突，单一综合分替代不了预算约束。这四件事分别对应本文在指标筛选、数据清洗、场景建模和工程决策四个环节的处理重点，后文依次展开。

1.2 统一数学对象

设共有 n 个模型配置、 p 个候选指标，原始矩阵为

$$X = (x_{ij})_{n \times p}, \quad (2)$$

其中 x_{ij} 表示第 i 个模型配置在第 j 个 **Benchmark** 上的观测。经过数据准入、正向化和分位数标准化得到 $Z = (z_{ij})$ ，再依据测量构念映射并融合为

$$C = (C_{ik})_{n \times 7}. \quad (3)$$

矩阵 Z 记录各 **Benchmark** 上的相对表现，矩阵 C 则按任务构念组织这些观测，并只对同构 **Benchmark** 作必要融合。本文不把该步骤解释为统计潜变量提取。问题一使用与场景无关的通用权重：

$$S_i^G = \sum_{k=1}^7 w_k^G C_{ik}. \quad (4)$$

问题二将场景 s 拆分为任务集合，通过任务重要度和评价维度依赖形成 W_s ，计算全部模型的场景综合得分：

$$S_{is} = \sum_{k=1}^7 w_{sk} C_{ik}. \quad (5)$$

相对短板约束只用于确定工程候选资格：

$$I_{is}(\theta) = \prod_{k \in K_s} \mathbf{1}(C_{ik} \geq \theta). \quad (6)$$

主模型固定 $\theta = 0.25$ 后，以下简记 $I_{is} = I_{is}(0.25)$ 。问题三在 S_{is} 和 I_{is} 的基础上加入价格与时延约束：

$$D_s(B) = \arg \max_i S_{is}, \quad \text{s.t.} \quad I_{is} = 1, \text{Cost}_{is} \leq B, T_i^{\text{bench}} \leq \tau_s. \quad (7)$$

1.3 总体技术路线

本文的计算流程为

$$X \xrightarrow{\text{准入、正向化、分位数标准化}} Z \xrightarrow{\text{语义映射与透明融合}} C \xrightarrow{\text{通用等权}} S^G \xrightarrow{\text{任务链需求与相对短板约束}} (S_{is}, I_{is}) \xrightarrow{\text{价格、时延与资源边界}} D_s(B). \quad (8)$$



图 1 从公开测评到场景决策的评价流程

2 数据、假设与符号

2.1 评价单元与数据来源

本文将评价单元定义为

$$u = (\text{模型版本, 推理档位, 工具配置, 评测协议}). \quad (9)$$

由于同一模型在不同推理档位或工具配置下，可能具有不同的 **Benchmark** 成绩、输出长度和响应时延，因此每个配置单独保留。公开质量成绩来自 **Artificial Analysis** 的统一测评页面，方法说明见 [1]。HLE、GPQA、Omniscience、AA-LCR、SciCode、Terminal-Bench 和 τ^3 -Banking 的任务定义及页面分别见 [2–16]。每个观测保留 **Benchmark** 名称、版本、分数单位、方向、评测日期、来源 URL、访问时间、证据路径和核验状态。

核心指标与扩展指标的覆盖率如下。覆盖率以 27 个质量面板为分母，扩展指标不通过插值进入主评价。

表 1 候选指标的准入与覆盖率

指标	类型	覆盖率	处理
HLE、GPQA Diamond	核心	100%	融合为 C_1
AA-Omniscience Accuracy	核心	100%	映射为 C_2
Non-Hallucination (由幻觉率转换)	核心	100%	映射为 C_3
AA-LCR	核心	100%	映射为 C_4
SciCode	核心	100%	映射为 C_5
Terminal-Bench v2.1	核心	100%	映射为 C_6
τ^3 -Banking	核心	100%	映射为 C_7
IFBench	扩展	40.7%	不进入主矩阵
LiveCodeBench	扩展	7.4%	不进入主矩阵
MMMU-Pro	扩展	51.9%	不进入主矩阵
Global-MMLU-Lite	扩展	7.4%	不进入主矩阵

在覆盖率阈值 **60%、70% 和 80%** 下，八项核心指标均通过，扩展指标均不能在 27 配置面板上通过。页面未展示的成绩记为缺失，不以 0、列均值或模型声誉代替。本文综合评价体系由七项主评价维度与低覆盖率扩展观察指标共同组成，主排名仅使用满足统一覆盖率和可比性要求的核心指标。

需要说明题目背景提及的多模态处理能力在本体系中的位置。候选指标中唯一的多模态观测为 MMMU-Pro，其覆盖率仅为 51.9%，低于 60%–80% 的全部准入阈值，按统一准入规则不得进入主矩阵；若为其单独放宽阈值，则不同模型进入评价的指标集合不再一致，可比性假设被破坏。因此本文七项核心评价维度均为文本类能力，多模态处理未纳入主评价体系，这是本文相对题目能力清单的已知覆盖缺口，本文全部排名与选型结论仅适用于文本类任务。若未来公开平台在统一协议下补足多模态观测，可按式 (3) 的同一映射框架将其增补为新的核心维度并重算，无需改动评价链的其余环节。

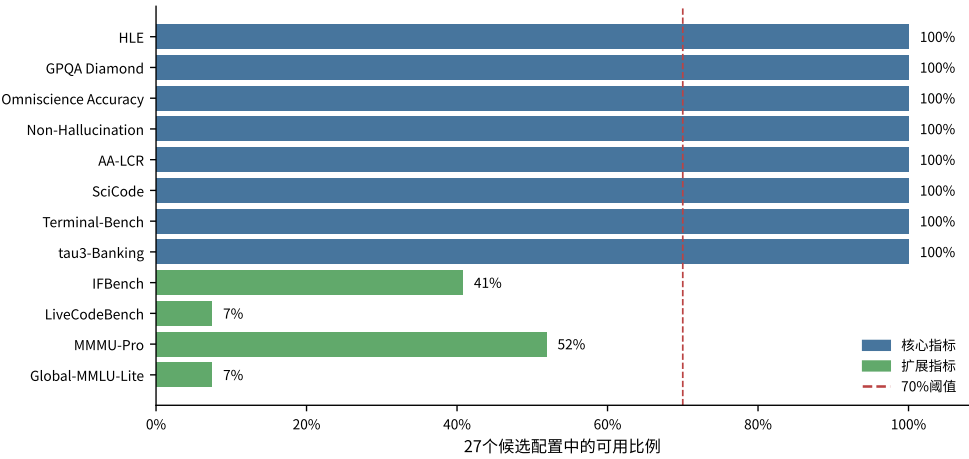


图 2 模型与候选 Benchmark 的覆盖率

图 3 列出 27 个质量完整配置在八项核心 Benchmark 上的原始观测，即矩阵 X 经准入筛选后的核心部分；数值与支撑材料 `results/core_observations.csv` 逐项一致，可与后续标准化结果对照核验。

原始核心矩阵 27 个完整质量配置									
仅含八项质量 Benchmark；数值原样来自 results/core_observations.csv。									
model_name	HLE	GPQA	Accuracy	NonHallucination	LCR	SciCode	Terminal	tau3	
Claude Fable 5 (with fallback)	55.5	92.6	65	36	76.7	60.2	84.6	38.1000	
Claude Opus 5 (max)	54.9	93.2	61	39	75.7	55.7	89.1	42.1000	
GPT-5.6 Sol (max)	49.5	94.1	59	8	77.7	56.1	88	44.3000	
Gemini 3.7 Flash (high)	47.9	94.5	55	35	80	56.8	85.8	32.8000	
Kimi K3 (max)	46.9	93.5	48	47	82.7	58.7	85	46.0000	
Muse Spark 1.2 (xhigh)	45.5	90.4	45	67	83.3	56.4	80.1	34.8000	
Qwen3.8 Max	43	92.7	32	58	74.3	52.9	81.3	51.3000	
GPT-5.6 Terra (max)	42.9	92.5	47	12	79.7	53.9	88	40.2000	
Grok 4.6 (high)	42.9	94.9	48	66	75	53.6	88.4	50.7000	
GLM-5.2 (max)	41.1	89.5	24	74	76.7	50.5	77.9	34.6000	
DeepSeek V4 Pro 0813 (max)	41	92.8	49	5	75.3	49.2	78.7	39.6000	
GPT-5.6 Luna (max)	39.5	91.1	43	7	78.3	52.5	80.9	31.1000	
MiniMax-M3	39	92.9	17	82	80.3	45.4	85.2	15.3000	
Motif 3	37	83.4	30	72	72.3	40.6	74.9	35.3000	
InKling	31.9	87.2	42	32	73.3	46.1	55.1	29.1000	
AX-K2	29.6	85.7	19	67	66	38.5	39	16.1000	
Solar Open2 250B	28.5	85.7	19	75	68.3	45.6	44.2	21.6000	
Nemotron 3 Ultra	28.4	86.7	23	70	71	39.9	53.9	14.2000	
Muse Glimmer (high)	22	83.5	27	18	80	43.6	51.7	23.5000	
Nemotron 3 Super	20.8	80	24	13	60.3	36	38.6	10.3000	
gpt-oss-120b (high)	19.6	78.2	22	9	51	38.9	26.2	12.8000	
Gemini 3.5 Flash-Lite	18.8	83.8	29	66	74.7	40.9	53.6	17.5000	
K-EXAONE 2.0	18.6	82.9	13	77	57.7	41	40.4	11.5000	
Mistral Medium 3.5	13.8	74.8	25	18	65.3	39.6	50.6	15.1000	
Command A+	12	76.1	9	86	48.7	37.8	22.8	6.0000	
Nemotron 3.5 Lightning	10.6	74.3	14	62	55.3	31.6	24.3	8.9000	
Claude 4.5 Haiku	10.4	67.2	18	73	73.7	43.3	44.2	9.3000	

图 3 27 个质量完整配置的八项核心 Benchmark 原始观测矩阵

2.2 数据准入与假设

假设一：同一平台、同一公开页面和同一冻结日展示的成绩具有可比评测协议；该假设不扩展到协议不明的厂商自报结果。

假设二: 冻结日价格代表该时点的可部署价格, 价格或模型版本更新后需重新计算。

假设三: 场景 token 规模代表标准化任务, 实际机构可以替换任务规模、调用次数和时延阈值。

假设四: T_i^{bench} 来自同一平台的统一评测协议, 只作为端到端响应效率代理; 它不等同于模型执行本文 $L/M/H$ 三档 token 工作负载时的场景响应时间。本文将数据字段 `end_to_end_response_time_seconds` 记为 T_i^{bench} , 单位为秒, 并保留 `provider_or_median_scope` 记录其平台中位数口径。

假设五: Output Tokens per Task 只反映公开测评中的相对生成侧负担, 不能转换为 GPU-hours、kWh 或完整推理计算量。

AA-Omniscience 页面将幻觉率 HR_i 定义为错误回答占“错误、部分回答和未作答”之和的比例 [7]。本文将其为同向观测

$$NH_i = 100 - HR_i. \quad (10)$$

该指标反映模型在未答对问题上的错误断言倾向, 不能解释为一般意义下“不产生幻觉”的概率。闭源配置缺少统一功率、硬件和运行时间观测, 本文不据此推断可比的实际电力消耗量。

2.3 主要符号

表 2 主要符号及含义

符号	含义
$X = (x_{ij})$	模型配置与候选 Benchmark 的原始观测矩阵
$Z = (z_{ij})$	正向化并分位数标准化后的相对得分矩阵
$C = (C_{ik})$	七项核心评价维度矩阵
C_{ik}	模型 i 在核心评价维度 k 上的得分
$W^G = (w_k^G)$	问题一的通用无偏好权重
p_{st}, a_{stk}	任务重要度、任务对评价维度的依赖强度
r_{sk}, w_{sk}	场景维度需求、场景评价权重
S_{is}, I_{is}	场景综合得分、相对短板资格变量
$Cost_{is}, T_i^{\text{bench}}$	直接费用、统一评测协议下的端到端响应时延代理
N_s, τ_s, h	统一比较调用次数、场景服务时延边界、缓存命中率假设
V_i, E_i^*	输出速度、生成侧负担代理

3 问题一：核心评价维度构建与通用评价

3.1 问题解析

问题一的输入是第 2 节筛选后的原始矩阵 X , 输出是可供三个场景共同使用的核心评价维度矩阵 C 和通用得分 S_i^G 。直接将所有 Benchmark 相加会混合量纲、重复构念和覆盖差异; 只选择少数榜单又会缩窄评价范围。

3.2 方法选择

本文先按任务构念确定观测归属, 再以 Spearman 秩相关和层次聚类提出潜在冗余候选, 最后进行正向化、分位数标准化和必要融合。Spearman 系数为

$$\rho_{jl} = \text{Corr}(\text{rank}(X_{\cdot j}), \text{rank}(X_{\cdot l})), \quad d_{jl} = 1 - \rho_{jl}. \quad (11)$$

所有质量观测已经正向化, 冗余预警统一使用 $\rho_{jl} \geq 0.85$ 。该阈值只提出潜在冗余候选, 不能直接决定删除; 本文依据统计关联、任务构念、覆盖率、区分度和来源可靠性共同决定合并或保留。在距离 $d_{jl} = 1 - \rho_{jl}$ 上, 程序采用 average linkage 进行凝聚型层次聚类。该方法按两簇样本对距离的平均值更新簇间距离, 兼顾簇的整体接近程度, 较少受单一最近点控制; 聚类树只用于显示相关结构和定位需要构念审查的指标群。实现使用 [17–19]。

表 3 给出统计预警如何转化为指标处理。HLE 与 GPQA 测量构念接近, 合并进入 C_1 ; Terminal-Bench 与 τ^3 -Banking 虽排序接近, 但分别侧重终端工程执行和交互式工具任务, 因而保留为 C_6, C_7 。

Accuracy 与 Non-Hallucination 的相关系数为负，且分别描述答对能力和知识边界校准，不属于正向冗余。表中只列具有代表性的高相关指标对；全部满足 $\rho \geq 0.85$ 的 8 组组合及逐项构念审查见支撑材料 all_high_corr_pairs.csv。

表 3 相关 Benchmark 的构念审查与处理

Benchmark A	Benchmark B	ρ	构念关系	实质冗余	处理方式	处理理由
HLE	GPQA	.9008	均侧重高难科学知识 与推理	是	融合为 C_1	避免同构观测重复计量
Terminal-Bench	τ^3 -Banking	.9035	工程执行与交互任务相 关但不同	否	分别保留	任务主体和交互对象不 同
SciCode	Terminal-Bench	.8925	科学代码与终端工程相 邻	否	分别保留	任务产物和执行环境不 同
HLE	Terminal-Bench	.8980	科学推理与工程执行排 序接近	否	分别保留	测量对象与任务产物不 同
Accuracy	Terminal-Bench	.8751	知识答对与工程执行相 关	否	分别保留	一个测知识正确性，一 个测环境执行

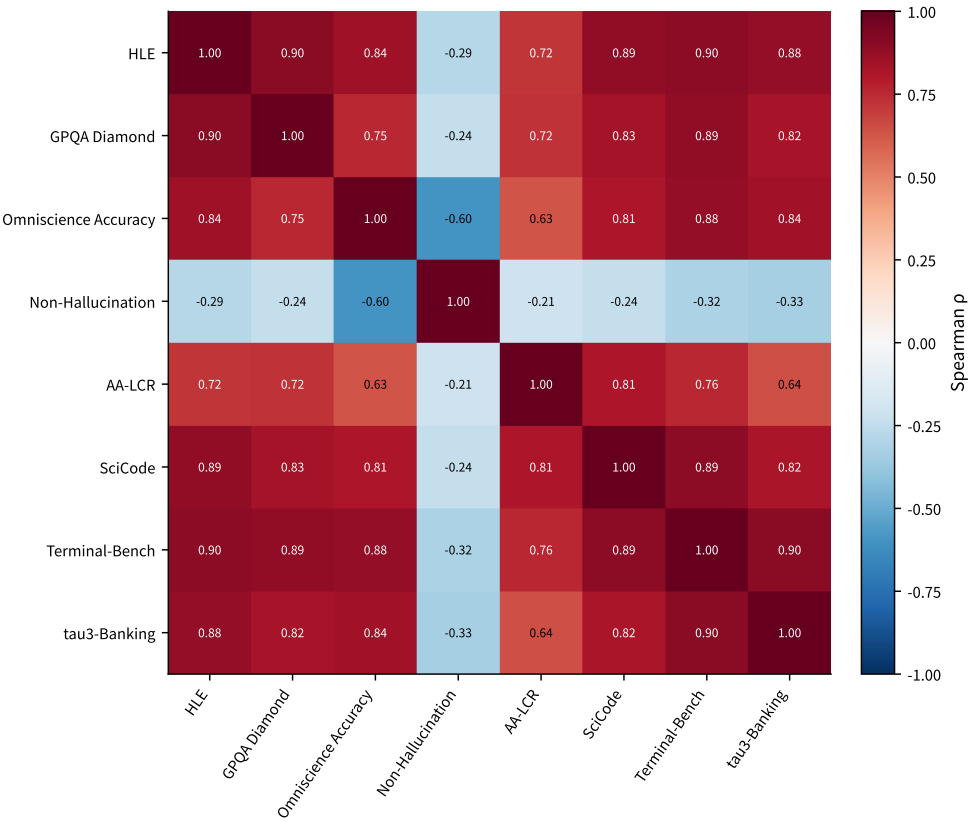


图 4 八项核心观测的 Spearman 相关矩阵

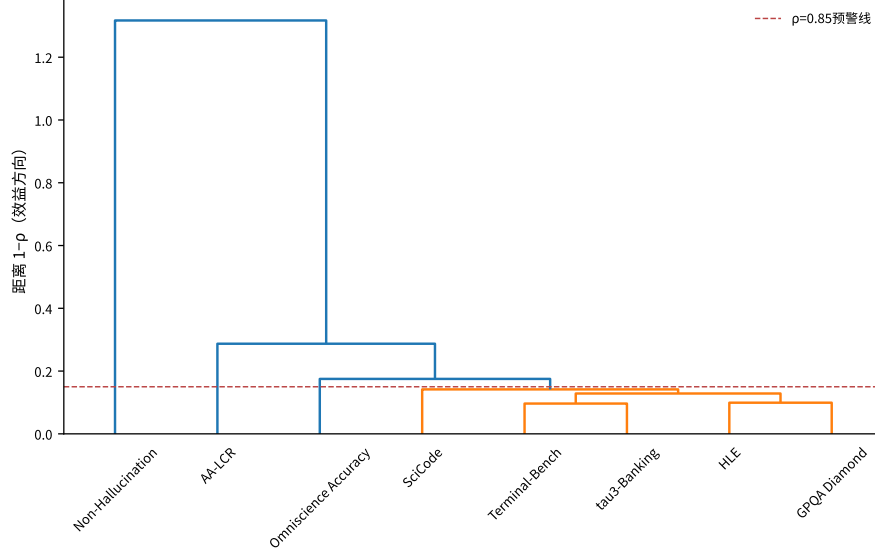


图 5 核心观测的层次聚类树

3.3 模型建立：七项核心评价维度

矩阵 C 用于组织 Benchmark 的任务含义，并为问题二提供统一接口。除 C_1 融合两项同构观测外，其余维度均保留单项透明映射，因此本文不将其称为潜变量。

表 4 七项核心评价维度与观测映射

维度	含义	主要观测	融合规则
C_1	科学与复杂推理	HLE、GPQA Diamond	两项等权
C_2	知识正确性	AA-Omniscience Accuracy	单项映射
C_3	知识边界校准	Non-Hallucination	单项映射
C_4	长文本理解	AA-LCR	单项映射
C_5	代码与科学计算	SciCode	单项映射
C_6	软件工程与工具执行	Terminal-Bench v2.1	单项映射
C_7	指令交互与任务完成	τ^3 -Banking	单项映射

对正向指标，分位数得分定义为

$$z_{ij} = \frac{\text{rank}(x_{ij}) - 1}{n_j - 1}, \quad 0 \leq z_{ij} \leq 1, \quad (12)$$

并列值采用平均秩。 z_{ij} 表示当前可比样本内的相对位置，不解释为能力达到概率。科学与复杂推理能力内

$$C_{i1} = \frac{z_{i,\text{HLE}} + z_{i,\text{GPQA}}}{2}. \quad (13)$$

其余维度取对应的单项标准化观测 $C_{ik} = z_{ij(k)}$ ($k = 2, \dots, 7$)。该透明映射避免在只有一或两项指标的维度内用样本离散度替代构念重要度。

3.4 模型求解：通用综合评价

问题一需要独立于三个场景。当前没有独立专家样本或可靠的通用任务频率先验，本文按最大熵原则构造无偏好基准 [20]：

$$\max_{W^G} H(W^G) = - \sum_{k=1}^7 w_k^G \ln w_k^G, \quad \text{s.t.} \quad \sum_{k=1}^7 w_k^G = 1, \quad w_k^G \geq 0. \quad (14)$$

由拉格朗日一阶条件得到 $w_k^G = 1/7$ ，故 $S_i^G = \sum_k w_k^G C_{ik}$ 。该解只表示缺乏可靠先验时的最弱偏好，不表示唯一合理的权重方案。程序读取质量长表，建立配置单元，核验八项核心指

标，转换 Non-Hallucination，计算分位数得分，合成 C ，最后计算 S_i^G 和完整排名。输出分别写入 ability_scores.csv、standardized_scores.csv 和 quality_mapping.csv。

3.5 结果分析：通用排名与 Kimi K3

表 5 报告 27 个质量完整配置的通用前五名，完整排名见支撑材料。

表 5 通用质量评价前五名

名次	模型配置	S_i^G
1	Kimi K3 (max)	0.824176
2	Claude Opus 5 (max)	0.793956
3	Grok 4.6 (high)	0.776099
4	Claude Fable 5 (with fallback)	0.771978
5	Gemini 3.7 Flash (high)	0.766484

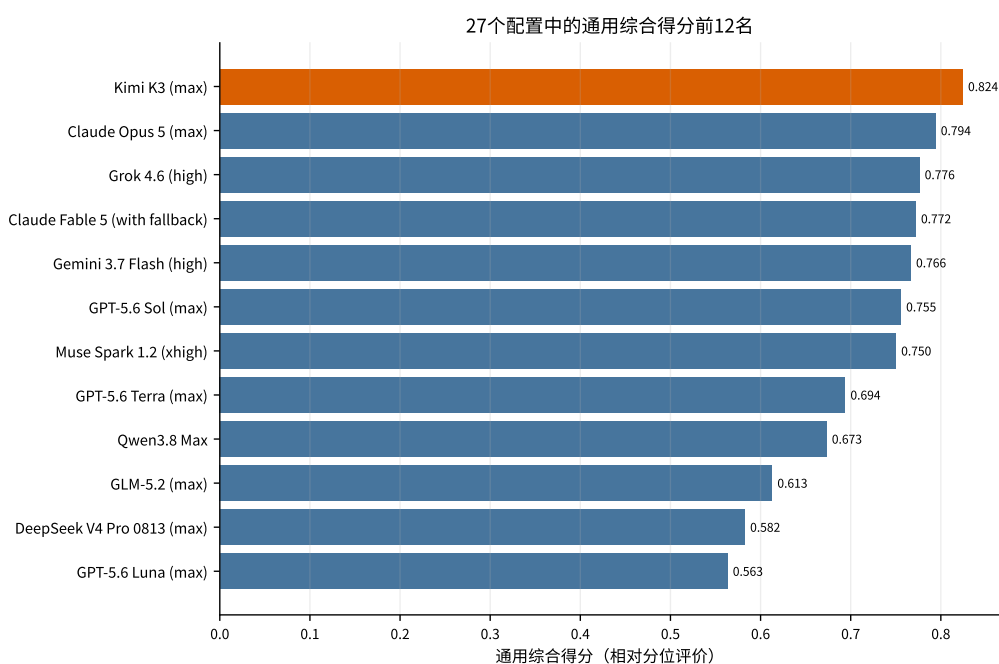


图 6 27 个配置中的通用综合得分前 12 名

Kimi K3 维度剖面。以样本均值为参照，定义 $A_k = C_{\text{Kimi},k} - \bar{C}_{\cdot,k}$ ，维度对通用分的贡献定义为 $G_k = w_k^G C_{\text{Kimi},k}$ 。

表 6 Kimi K3 的核心评价维度分解

维度	得分	相对均值	通用贡献	与该项最高者 差距
C_1 科学推理	0.865385	0.365385	0.123626	0.057692
C_2 知识正确性	0.788462	0.288462	0.112637	0.211538
C_3 知识边界校准	0.461538	-0.038462	0.065934	0.538462
C_4 长文本理解	0.961538	0.461538	0.137363	0.038462
C_5 代码与科学计算	0.961538	0.461538	0.137363	0.038462
C_6 软件工程与工具执行	0.807692	0.307692	0.115385	0.192308
C_7 指令交互与任务完成	0.923077	0.423077	0.131868	0.076923

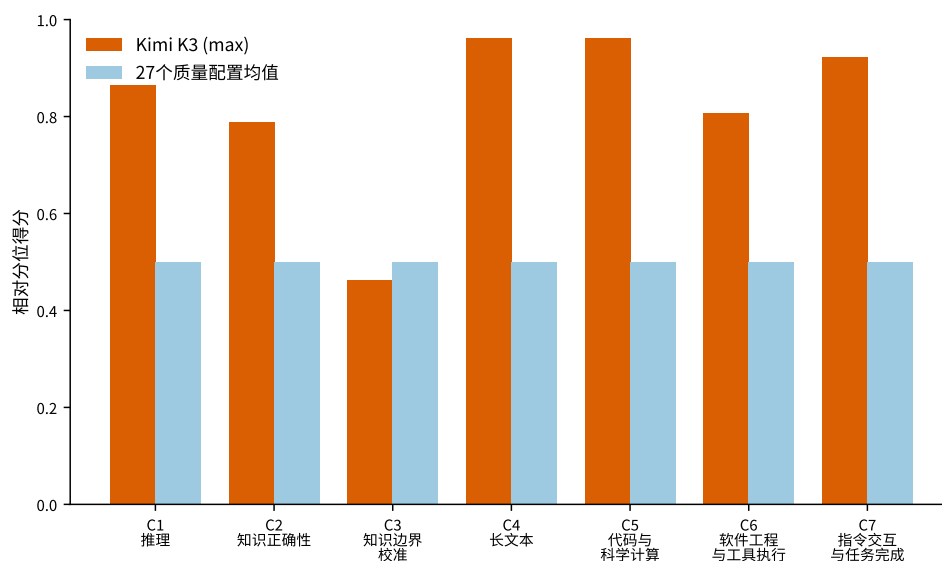


图 7 Kimi K3 的七项核心评价维度剖面

Kimi 在长文本、科学代码和交互任务三个维度上均高于样本均值，与对应最高者的差距不超过 0.076923。知识边界校准得分为 0.461538，低于样本均值 0.5；该值只说明 Kimi 在未答对问题时避免错误断言的相对表现较弱，不表示一般意义下的幻觉概率。其通用首名由多个维度共同贡献，并非来自单项边界校准领先。

赛题指定的官方资料 [21] 将 Kimi K3 的能力重点表述为百万级 token 原生上下文、长时程代码开发和智能体任务执行。该资料属于厂商自报口径，按第 2 节假设不作为测评数据来源，此处仅用于定性对照。其能力表述与本文基于第三方统一测评的独立结果相互印证：本文中 Kimi K3 的相对优势恰集中于长文本理解、代码与科学计算、软件工程与工具执行三个维度；该资料同时承认其整体表现仍落后于个别最强闭源模型，与本文发现的知识边界校准相对短板共同界定了 Kimi K3 的改进方向。两类来源在优势与短板的具体位置上结论一致，说明本文的评价结果不是单一平台数据的偶然产物。

3.6 本问小结

问题一将八项核心 Benchmark 组织为七项核心评价维度，形成 C 。相关性和聚类提出冗余候选，构念审查决定合并或保留；HLE/GPQA 等权融合，其余维度采用单指标透明映射。最大熵无偏好权重下，Kimi K3 (max) 在 27 个质量完整配置中以 0.824176 列第 1。矩阵 C 将原样传递给问题二。

4 问题二：任务链驱动的场景评价

4.1 问题解析

问题一给出了固定的评价向量 C_i 。问题二不重新处理 **Benchmark**，而是考察不同场景如何改变七项维度的重要度。

4.2 方法选择：任务链与事前编码

同一评价向量在三个场景中的价值不同，本文先把每个场景拆为五项可辨认任务，再编码任务对各评价维度的依赖。编码遵循统一准则：0 表示该维度与任务基本无关，缺失不影响主体完成；1 表示辅助依赖，只影响局部效率或结果细节；2 表示重要依赖，对主要质量有明显影响；3 表示核心依赖，缺失时任务难以完成或结果受到显著破坏。该矩阵来自题设场景和任务流程的事前结构化编码，不声称来源于专家问卷、AHP 或实证频率。

五任务链按典型工作流程作功能分解，依次覆盖输入理解或信息定位、核心分析或生成、信息整合或执行、验证或调试、输出组织与任务完成。各任务在功能上可区分，合计覆盖主要工作阶段，但允许依赖相同评价维度；该划分只用于形成可复核的场景需求，不声称是唯一分类。科研链覆盖文档定位、跨文档综合、科学推理、证据核验和报告组织；日常对话链覆盖意图理解、事实问答、多轮跟进、规划推理和答复核验；代码链覆盖需求设计、代码生成、理解重构、调试测试和终端执行。

4.3 模型建立：场景权重

题目没有提供五项子任务的发生频率、失败成本或重要度统计。为保持与问题一一致的无偏好原则，在每个场景内求解

$$\max_{p_s} - \sum_{t=1}^5 p_{st} \ln p_{st}, \quad \text{s.t.} \quad \sum_{t=1}^5 p_{st} = 1, p_{st} \geq 0,$$

得到 $p_{st} = 1/5$ 。该值只是在缺少任务频率、失败成本和重要度先验时的基准设定，不是任务发生频率的实证估计。表 7 以 $(C_1, \dots, C_7)$ 向量给出完整依赖参数。

表 7 三类场景任务链与评价维度依赖向量

场景	任务	依赖向量 $(C_1, \dots, C_7)$
科研长文本	信息定位	(1,2,2,3,0,0,1)
	跨文档综合	(2,2,2,3,0,0,1)
	科学推理	(3,2,2,1,1,0,0)
	证据核验	(2,2,3,2,0,0,0)
	报告组织	(1,1,2,1,0,0,2)
大众日常对话	意图理解	(1,1,1,0,0,0,3)
	事实问答	(1,3,3,0,0,0,1)
	多轮跟进	(1,1,2,1,0,0,3)
	规划推理	(2,1,1,0,0,0,2)
	答复核验	(1,1,3,0,0,0,2)
代码开发	需求设计	(2,1,1,1,2,1,1)
	代码生成	(2,1,1,1,3,1,0)
	理解重构	(2,1,1,2,3,2,0)
	调试测试	(3,1,2,1,3,3,0)
	终端执行	(1,0,1,1,2,3,1)

代表任务的编码核验。科研场景的“信息定位”编码为 (1, 2, 2, 3, 0, 0, 1)：长文本理解直接决定能否在长文档中定位证据，取 3；知识正确性和边界校准影响检索结果的判断与引用，取 2；复杂推理和交互组织只起辅助作用，取 1；代码及终端执行不属于任务主体，取 0。日常场景的“事实问答”编码为 (1, 3, 3, 0, 0, 0, 1)：答对知识与避免错误断言共同决定答复质量，取 3；推理和交互影响解释过程，取 1；其余维度不直接参与。代码场景的“调试测试”编码为 (3, 1, 2, 1, 3, 3, 0)：推理、代码和终端执行决定定位并验证缺陷，取 3；知识边界校准用于抑制未经验证的修复判断，取 2；知识正确性和长文本理解提供辅助，取 1；通用对话交互不构成主体，取 0。三组示例使用同一尺度，便于复核其余任务行。场景需求和权重分别为

$$r_{sk} = \sum_t p_{st} a_{stk}, \quad w_{sk} = \frac{w_k^G r_{sk}}{\sum_{h=1}^7 w_h^G r_{sh}}. \quad (15)$$

表 8 场景需求向量

场景	C_1	C_2	C_3	C_4	C_5	C_6	C_7
科研长文本	1.800	1.800	2.200	2.000	0.200	0	0.800
日常对话	1.200	1.400	2.000	0.200	0	0	2.200
代码开发	2.000	0.800	1.200	1.200	2.600	2.000	0.400

表 9 场景评价权重

场景	C_1	C_2	C_3	C_4	C_5	C_6	C_7
科研长文本	.204545	.204545	.250000	.227273	.022727	0	.090909
日常对话	.171429	.200000	.285714	.028571	0	0	.314286
代码开发	.196078	.078431	.117647	.117647	.254902	.196078	.039216

4.4 模型求解：场景得分、相对短板约束与贡献分解

场景综合得分为 $S_{is} = \sum_k w_{sk} C_{ik}$ ，用于 27 个配置的完整场景排名。为防止线性加权让非核心优势完全补偿核心维度短板，科研、日常和代码的核心维度索引集合分别为

$$K_R = \{1, 3, 4\}, \quad K_D = \{2, 3, 7\}, \quad K_C = \{1, 5, 6\}.$$

主值取 $\theta = 0.25$ 。由于 C_{ik} 是 27 个候选配置上的分位数得分，该值表示模型在场景核心维度上不得落入当前比较集合的后约四分之一。需要强调，分位数得分是相对排名坐标而非绝对能力刻度：任一维度排名最后的配置其 C_{ik} 恰为 0，会被 $\theta = 0.25$ 自动剔除，但这只说明它在当前 27 个可比配置中处于该维度末位，不指控其绝对能力为零；候选集合扩大后，原末位配置的分位数将上升，资格需重新判定。式 (6) 因而是相对短板约束，不是现实业务中的绝对能力标准；候选集合变化后需重新计算。两个场景间的得分变化可分解为

$$\Delta S_i^{a \rightarrow b} = \sum_{k=1}^7 (w_{bk} - w_{ak}) C_{ik}. \quad (16)$$

相对短板约束造成的资格变化单独报告，不计入线性贡献。

4.5 结果分析：三场景排名

表 10 给出三类场景前五名，均为 27 个质量完整配置上的 S_{is} 。为排版紧凑，表中模型名采用简称：Kimi 指 Kimi K3 (max)，Opus 指 Claude Opus 5 (max)，Fable 指 Claude Fable 5 (with fallback)，Muse Spark 指 Muse Spark 1.2 (xhigh)，Gemini Flash 指 Gemini 3.7 Flash (high)，GPT Sol 指 GPT-5.6 Sol (max)，Grok 指 Grok 4.6 (high)，Qwen 指 Qwen3.8 Max。

表 10 三类场景综合得分前五名

场景	第 1 名	第 2 名	第 3 名	第 4 名	第 5 名
科研长文本	Kimi .777972	Muse Spark .758304	Gemini .726399	Flash Opus .722465	Fable .715909
日常对话	Grok .792308	Kimi .755495	Opus .751648	Qwen .714286	Fable .703846
代码开发	Kimi .838612	Gemini .816742	Flash Opus .809955	Fable .803167	GPT Sol .776018

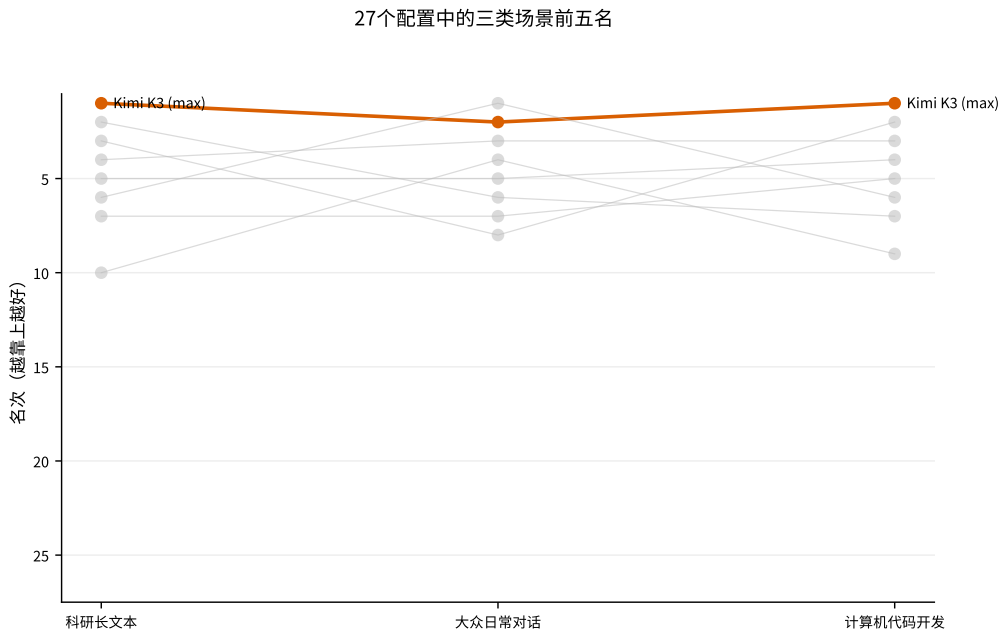


图 8 27 个配置中的三类场景综合得分前五名

在 $\theta = 0.25$ 的相对短板约束下，科研、日常和代码的资格数分别为 15、13 和 18。Kimi K3 的三场景得分、名次和资格为：科研 0.777972、1/27、通过；日常 0.755495、2/27、通过；代码 0.838612、1/27、通过。

以 Kimi 为例，从科研到日常的得分变化为 -0.022477 ，其中 C_4 长文本权重变化贡献 -0.191059 ， C_7 交互任务权重变化贡献 $+0.206194$ ，其余维度合计 -0.037612 。从科研到代码， C_5 、 C_6 权重增加分别贡献 $+0.223245$ 、 $+0.158371$ ，抵消其他维度权重下降，最终得分增加 $+0.060640$ 。可见 Kimi 的场景适应性来自维度结构的匹配，而非单项能力突变。

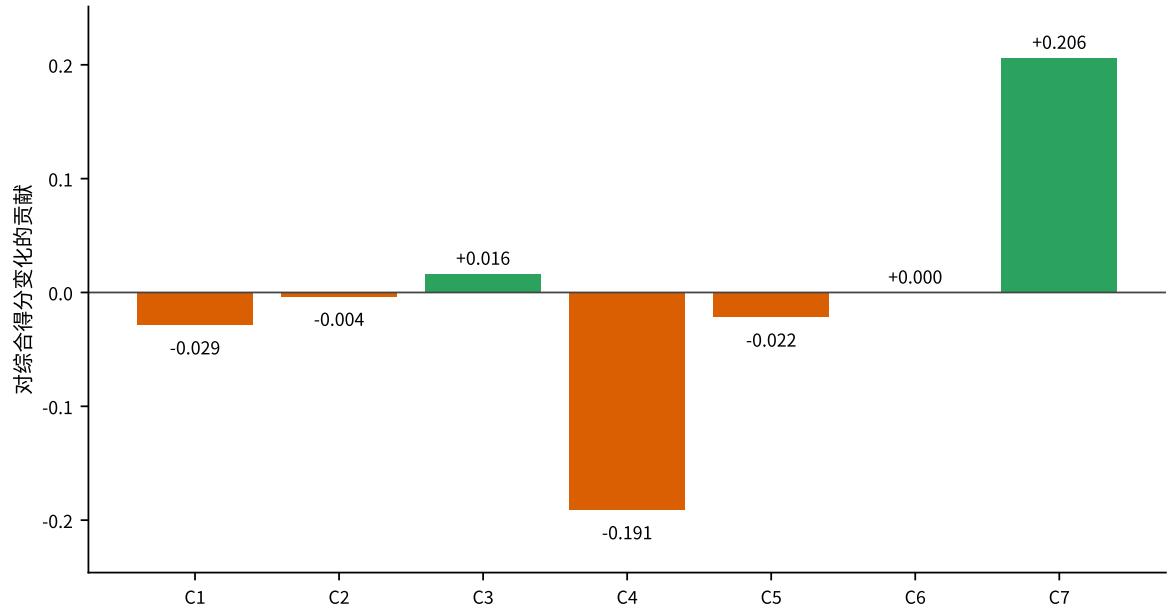


图 9 Kimi K3 的跨场景维度贡献分解

4.6 本问小结

问题二保持 C 不变，由“场景-任务-评价维度”链条推导 W_s ，再得到 27 个配置的场景综合得分。科研场景首名为 Kimi，日常首名为 Grok，代码首名为 Kimi；相对短板约束只改变工程候选集合，不改变场景排名。问题三进一步加入价格、时延和可部署边界。

5 问题三：性能-成本权衡与工程选型

5.1 问题解析

问题三以 S_{is} 为场景质量目标，以 $I_{is} = 1$ 表示满足相对短板约束，再加入费用和服务边界。场景得分高的配置未必低价或低时延，质量排名不能直接替代工程选择。

5.2 方法选择：质量、费用与时延分开决策

为避免把不同含义的工程量压缩成一个“成本”，分别记录 P_i^{in}, P_i^{out} （输入、输出 token 单价）、 T_i^{bench} （统一评测协议下的端到端时延代理）、 V_i （输出速度）和 E_i^* （生成侧负担代理）。由于缺少所有模型在本文三档标准工作负载下的统一时延实测，主 Pareto 只把 T_i^{bench} 用于形成相对工程可行域，不把它解释为指定 token 任务的场景响应时间。若未来取得口径一致的 TTFT 和输出速度，可用 $\hat{T}_{is} = TTFT_i + L_s^{out}/V_i$ 扩展估计；当前数据协议不能保证一致，故该式不进入主结果。

为避免将情景假设与外部观测混为一谈，本文进一步区分三类变量。第一类为决策者给定的外生情景参数，包括任务输入、输出长度 L_s^{in}, L_s^{out} 、统一比较调用次数 N_s 和服务时延边界 τ_s ，用于描述待部署业务的任务规模与服务要求；第二类为公开可观测的模型工程数据，包括输入、输出 token 单价 P_i^{in}, P_i^{out} 、统一评测协议下的端到端时延代理 T_i^{bench} 和输出速度 V_i ；第三类为模型根据前两类变量计算得到的内生结果，包括直接费用 $Cost_{is}$ 、Pareto 有效集、预算切换节点和最终选型 $D_s(B)$ 。本文设定的任务规模与时延边界属于可替换的决策情景，不解释为行业统一统计值。

输出速度 V_i 描述模型开始生成后的 token 吞吐能力，与统一协议端到端时延从不同角度反映服务效率。考虑到两者存在信息重叠，且 T_i^{bench} 已进入主 Pareto 模型，为避免对响应效率重复计量，本文不再将 V_i 单独作为主优化目标，仅将其用于长输出任务的补充工程观察。

5.3 模型建立：工作负载与直接费用

标准化工作负载设定。为使不同模型的价格具有统一可比的计算基础，本文为三类场景构造轻量 L 、标准 M 和重度 H 三档代表性工作负载。这里的 token 数属于情景参数，其功能是模拟不同业务规模下的调用需求，不作为现实业务平均长度的统计估计。对科研长文本场景，本文以 C_4 维度所用 AA-LCR 长上下文测评的任务规模为数量级参照 [8, 9]，取 80k 输入、4k 输出构造标准负载，并在其两侧设置较轻和较重情景，覆盖不同规模的文献、报告和多文档分析任务；对代码开发场景，代码理解、重构和智能体工作流需同时处理需求、代码上下文和执行反馈，故采用数万 token 量级作为代表性输入规模；日常对话缺乏统一的权威任务长度统计，本文采用较短上下文和较短输出构造普通交互情景，使其与长文本和代码场景形成明显规模差异。三档负载的主要作用是考察任务规模变化是否改变费用水平、Pareto 前沿和预算选型路径；实际应用中，决策者可将表 12 中的 L_s^{in} 和 L_s^{out} 替换为本机构真实工作负载，无需改变后续模型结构。表 11 汇总第三问全部变量的来源与作用。

表 11 第三问参数的来源与作用

变量	类型	当前主值	含义与依据
L_s^{in}, L_s^{out}	外生情景参数	见表 12	构造代表性任务规模，科研档以 AA-LCR 任务长度校准数量级
N_s	外生情景参数	1,000 次	统一费用比较尺度，可替换
τ_D, τ_C, τ_R	外生决策参数	30/90/180 s	日常、代码、科研的服务容忍边界情景，非行业统一 SLA
θ	模型结构参数	0.25	相对短板阈值，敏感性分析见 6.1 节
h	保守假设	0	缓存命中率，只影响费用，不影响质量得分
P_i^{in}, P_i^{out}	外部观测数据	冻结日价格	用于计算实际调用费用
T_i^{bench}	外部观测数据	平台数据	统一协议下的端到端响应效率代理
V_i	外部观测数据	平台数据	输出速度，仅作扩展工程观察
β, η	假设值	64、0.4	并发批大小、算力利用率，仅用于能耗扩展，做敏感性分析
λ_E, PUE	行业公开值	0.6 元/kWh、1.25	工业电价代表值、机房能效，仅用于能耗扩展
$Cost_{is}$	模型输出	计算得到	场景工作负载下的直接调用费用
预算节点 B_j	模型输出	计算得到	最优推荐发生变化的预算切换点

情景参数用于构造可替换的业务环境，公开观测用于描述模型实际工程属性，预算节点和推荐路径则由模型自动计算。

表 12 三类场景的标准化工作负载（token 数为标准化决策情景，非行业平均值，可整体替换）

场景	轻量 L	标准 M	重度 H
科研长文本	20k 输入 +2k 输出	80k 输入 +4k 输出	160k 输入 +8k 输出
日常对话	1k 输入 +0.3k 输出	2k 输入 +0.8k 输出	8k 输入 +2k 输出
代码开发	5k 输入 +1k 输出	20k 输入 +4k 输出	50k 输入 +8k 输出

设单次任务输入、输出 token 数为 L_s^{in}, L_s^{out} ，模型输入、输出单价为 P_i^{in}, P_i^{out} ，统一比较调用次数为 $N_s = 1,000$ ，则 N_s 次任务的直接费用为

$$Cost_{is} = N_s \cdot \frac{L_s^{in} P_i^{in} + L_s^{out} P_i^{out}}{10^6}. \quad (17)$$

本文固定缓存命中率 $h = 0$ 。Command A+ 因缺少可解释的零成本部署证据不视为免费配置，排除出 20 个正价成本面板。

5.4 模型建立：Pareto 与预算选择

模型 A 支配模型 B，当且仅当 A 的质量不低于 B、费用不高于 B、时延不高于 B，且至少一项严格更优。主 Pareto 前沿为

$$(S_{is} \uparrow, Cost_{is} \downarrow, T_i^{bench} \downarrow), \quad I_{is} = 1, \quad (18)$$

其定义与 [22] 一致。给定预算 B 和时延上限 τ_s ，选型规则为

$$D_s(B) = \arg \max_i S_{is}, \quad \text{s.t.} \quad I_{is}(0.25) = 1, Cost_{is} \leq B, T_i^{bench} \leq \tau_s. \quad (19)$$

5.5 模型求解与结果分析：中等负载预算路径

程序分别在 L, M, H 三档负载下，对 20 个价格字段可核验且为正的配置计算费用，读取 S_{is} 和 I_{is} ，先筛除不满足相对短板约束或时延边界者，再计算 Pareto 前沿和预算分段。预算区间采用左闭右开约定，节点取可行模型实际费用的排序切点。

服务时延情景。时延上限 τ_s 视为决策者根据业务交互需求给定的服务容忍边界，而非由模型性能数据估计得到的统计参数。三类场景的交互同步性存在明显差异：日常对话是同步轮流交互，等待时间直接构成体验损失；代码开发是半同步的迭代调试，允许模型进行一定程度的分析与执行，但仍需较快反馈；科研长文本分析接近异步批处理任务，提交后等待属于可接受的工作模式，对单次响应时间的容忍度相对更高。因此本文令 $\tau_D < \tau_C < \tau_R$ ，并分别以 30 s、90 s、180 s 作为日常、代码和科研场景的基准服务情景。上述数值用于构造不同实时性要求下的工程可行域，不解释为行业统一 SLA；6.3 节通过改变时延边界检验最终推荐对该设定的敏感程度。

预算节点的内生来源。与工作负载和时延边界不同，表 13 中的预算切换节点并非事先指定的模型参数。本文首先根据场景工作负载和冻结日公开输入、输出单价计算每个模型配置的 $Cost_{is}$ ，随后在满足相对短板约束和时延边界的配置中，按场景综合得分寻找各预算水平下的最优配置；随着预算 B 增加，当一个综合得分更高的配置首次进入预算可行域时，即形成新的预算切换节点。因此 28.8、184、300 等数值属于当前情景参数与冻结价格共同作用下的模型内生结果，而非人为预先设定的预算等级。表 13 的预算单位为美元/1,000 次任务，所有节点只适用于本文中等标准负载、冻结日价格和既定时延边界。

表 13 中等负载预算约束下的分段选型路径

场景	预算区间	推荐配置（场景综合得分）
科研长文本	[0, 28.8) [28.8, 184) [184, 300) [300, +∞)	无可行模型 MiniMax-M3 (0.649913) Grok 4.6 (0.711757) Kimi K3 (0.777972)
日常对话	[0, 1.72) [1.72, 2.60) [2.60, 6.32) [6.32, +∞)	无可行模型 Muse Glimmer (0.386264) Gemini 3.5 Flash-Lite (0.451648) GLM-5.2 (0.629670)
代码开发	[0, 10.8) [10.8, 45.6) [45.6, 64) [64, 120) [120, +∞)	无可行模型 MiniMax-M3 (0.590498) GLM-5.2 (0.618401) Grok 4.6 (0.775641) Kimi K3 (0.838612)

中等负载预算路径。在本文中等标准负载、冻结日价格和既定时延边界下，Kimi K3 的直接费用分别为科研 300、日常 18、代码 120 美元/1,000 次任务。科研任务预算达到 300 美元时，Kimi K3 进入场景得分最高的可行方案。日常预算路径的上限段由 GLM-5.2 承接，这是当前相对短板约束、时延和价格可行域下的结果，不代表 Kimi 的日常得分低于 GLM。

此处需要解释表 10 与表 13 之间的表面差异。日常场景综合得分第 1、2 名的 Grok 4.6 (0.792308) 和 Kimi K3 (0.755495) 均未出现在日常预算路径中：二者在预算分配之前的可行域筛选阶段已被排除。其中 Kimi K3 的日常相对短板资格通过，且直接费用 18 美元/1,000 次任务处于可负担区间，其缺席只能归因于统一协议时延代理超过日常场景 30 s 的时延上限；Grok 4.6 同样未通过日常场景的资格或时延筛选，逐配置的约束核验记录见支撑材料。这一现象本身值得点出：高场景得分配置在强交互、低时延业务中可能整体不可行，预算路径的顶端只能由同时满足全部约束的最高得分配置承接。

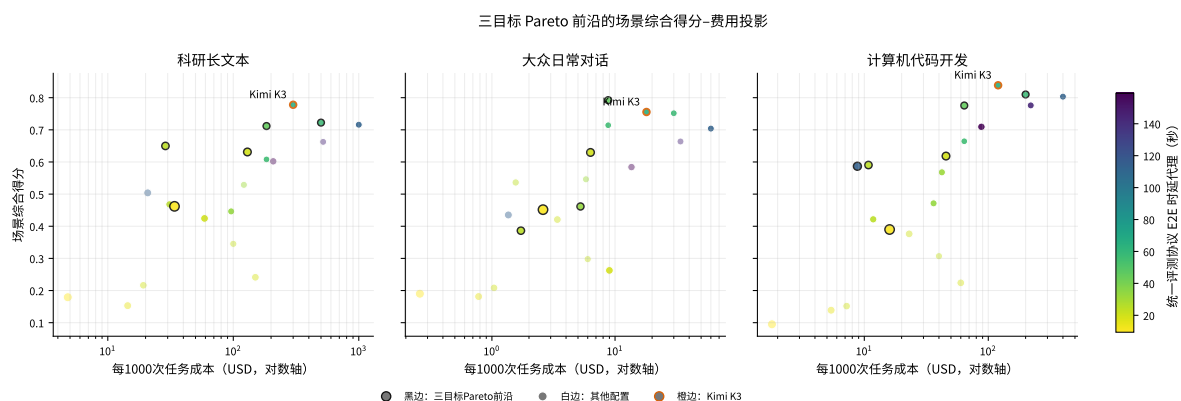


图 10 三目标 Pareto 前沿的场景综合得分-费用投影

图 10 只把三目标前沿投影到场景综合得分-费用平面；前沿成员仍由质量、费用和统一协议时延代理共同确定，点的颜色表示 T_i^{bench} 。

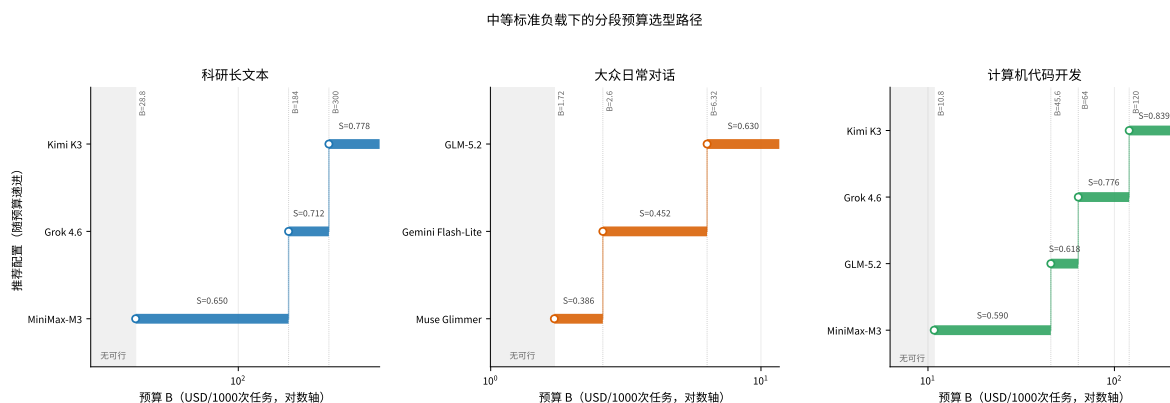


图 11 三类场景的预算切换路径

5.6 结果分析：成本–质量关系与工作负载敏感性

程序比较线性、对数、幂函数、饱和和单断点模型，以留一交叉验证误差为首要选择依据 [23]。表 14 记录冻结样本内的探索性拟合，不预设统一函数形式。

表 14 成本–质量函数的选择与拟合结果

场景	入选函数	LOOCV RMSE	调整 R^2	Spearman ρ
科研长文本	单断点, 结点 $B = 300$	0.092462	0.431390	0.669706
日常对话	对数	0.161157	0.191335	0.510641
代码开发	饱和, $\tau = 68.134$	0.092882	0.643919	0.842685

科研和代码场景的 Spearman 系数分别为 0.669706 和 0.842685，说明冻结样本内费用与场景得分存在正向关联；日常场景的调整 $R^2 = 0.191335$ ，成本解释力较弱。三类场景分别选出单断点、对数和饱和形式，但留一模型检验中科研单断点仅 5/11 次保持、代码饱和形式 9/14 次保持，具体函数受少数配置影响。表中 $B = 300$ 只是本文中等标准负载和冻结样本下入选单断点模型的探索性拟合结点，不是普适预算阈值；本文也不作价格导致性能提高的因果推断。轻、中、重负载下的质量–费用–时延三目标 Pareto 集保持一致：科研、日常和代码前沿规模分别为 6、5、7；中等负载与轻量、重度负载的 Jaccard 指数均为 1。

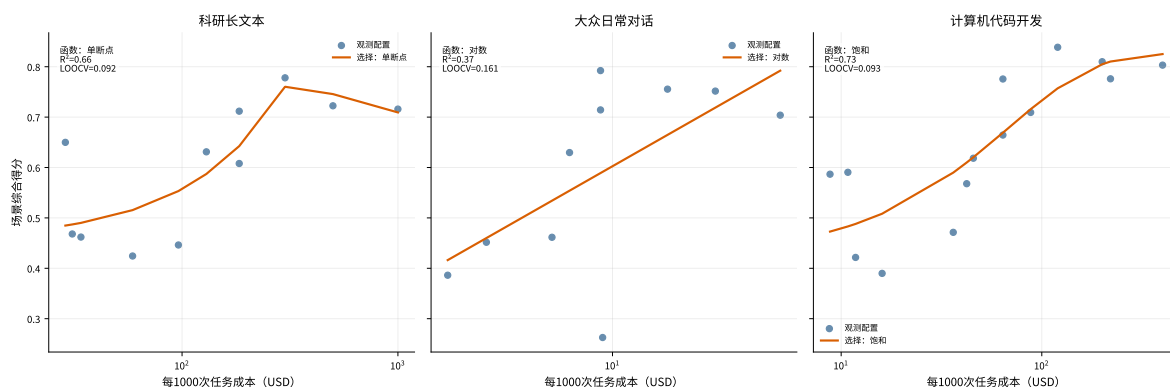


图 12 三类场景的成本–质量拟合

生成侧计算负担代理。不同模型即使完成相同 Benchmark，其平均输出 token 数仍可能存在明显差异，该差异在一定程度上反映模型完成任务所使用的生成长度和推理展开程度。本文使用同口径 Benchmark 上的 Output Tokens per Task 构造生成侧相对负担代理 E_i^* ，用于比较“达到相近任务质量时，模型需要产生多少输出 token”。但输出 token 数同时受回答风格、推理模式和任务格式影响，不能直接转换为 FLOPs、GPU-hours 或 kWh，公开平台也未提供同口径的 GPU-hours 或 kWh 观测。当前仅 HLE、GPQA 两个核心基准能获得统一可比的单任务输出 token 数据（覆盖 2/8 个核心基准），

因此 E_i^* 不进入质量排名和主预算模型，仅作为扩展工程观察；图 13 不能解释为统一口径的电力消耗量。当未来获得更完整的同口径 token 使用或真实硬件能耗数据时，可将该代理替换为实际计算资源指标，而不改变本文的多目标决策框架。若未来获得统一硬件、批量、功率和推理时间条件下的单任务能耗 E_{is} ，可定义扩展落地成本

$$TC_{is} = Cost_{is} + \lambda_E E_{is},$$

其中 λ_E 可表示电价、计算资源影子价格或碳成本；也可在 $(S_{is} \uparrow, Cost_{is} \downarrow, T_i^{\text{bench}} \downarrow, E_{is} \downarrow)$ 上重算四目标 Pareto 前沿。现有数据不满足统一口径，本文只保留接口，不作数值求解。

能耗接口的参数化实例。闭源配置的能耗因参数与硬件不公开而无法估算；对开源配置，可用公开硬件参数作第一性原理估计，把上式接口实例化。自回归推理中，prefill 阶段受算力约束、decode 阶段受显存带宽约束，单任务能耗可写为

$$E_{is} = \underbrace{\frac{2N_{act}L_s^{in}}{\eta F} \cdot P}_{\text{prefill}} + \underbrace{\frac{B_{act}}{BW} \cdot P \cdot \frac{L_s^{out}}{\beta}}_{\text{decode}},$$

其中 N_{act} 为激活参数量， B_{act} 为单步需读取的激活权重字节数， F 与 BW 为加速卡有效算力与显存带宽， P 为功耗， β 为并发批大小， η 为算力利用率。以开源的 Kimi K3 为例：官方资料给出总参数 2.8T、896 专家激活 16、MXFP4 权重量化 [21]，激活参数量约 50B、单步激活权重约 25GB；加速卡取 H100 SXM（TDP 700W、带宽 3.35TB/s，官方 datasheet），电价取大陆工业平时段代表值 0.6 元/kWh、PUE 取 1.25， $\beta = 64$ 、 $\eta = 0.4$ 为假设值（数据来源与敏感性区间见支撑文件）。代入科研标准负载（80k 输入 +4k 输出）得单任务能耗约 0.0026 kWh、电费约 0.0015 元——prefill 占绝对主导，且电费相对 API 费用（约 0.3 美元/次）不足千分之一。

这一结果说明：私有化路径的可变能耗成本极低，真正决定自建是否划算的是硬件固定成本而非电费。按 64 卡 H100 全托管约 2 美元/卡·时估算日固定成本约 3072 美元，以科研 API 单次 0.30 美元计，盈亏平衡日调用量约 $Q^* \approx 1.0$ 万次（区间 0.6 万-1.5 万次，随硬件价与负载档变化）。结论为：日调用量低于万次级宜走 API，高于该量级且有数据合规要求时才考虑自建。能耗并非私有化的主要成本项，硬件固定成本才是。该估计仅为量级刻画，不改变表 13 的主预算路径。

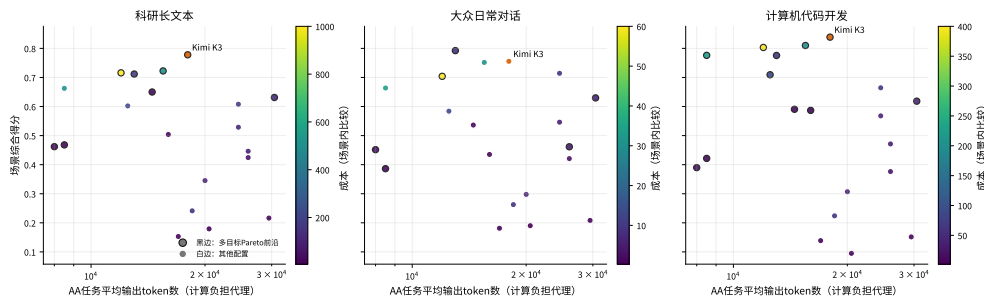


图 13 生成侧负担代理的扩展 Pareto 观察

5.7 本问小结

问题三将场景得分、相对短板资格、直接费用和统一协议时延代理分离建模。在 20 个正价配置上，主 Pareto 同时考察 S_{is} 、费用和 T_i^{bench} ；预算推荐先筛选不满足相对短板或时延边界的配置，再选择场景得分最高者，输出 Pareto 集与 $D_s(B)$ 。三类场景在本文中等负载下均呈现分段选型路径，节点随任务长度、价格和服务边界改变。输出速度和生成侧 token 代理仅作扩展观察。

综上，问题三构建的是一套参数化工程选型模型：任务 token 规模、调用次数和服务时延边界由决策者作为外生情景输入，模型价格和统一协议工程指标来自冻结公开数据，直接费用、Pareto 有效集、预算切换节点和最终推荐由模型内生计算得到。本文给出的具体预算路径仅对应当前标准化情景，其核心价值在于提供一套可由实际机构替换任务规模和服务要求后重新求解的决策方法。

6 模型检验、敏感性与稳健性分析

6.1 参数敏感性

本节只保留会影响模型解释或工程推荐的检验，完整参数、逐配置结果和随机抽样明细见支撑材料。程序对数据规模、权重归一、相对短板约束和预算可行域执行 33 项检查，均与模型定义一致；这类检查验证计算实现，不替代公开 Benchmark 的测量误差分析。

Benchmark 准入与场景编码。覆盖率阈值取 60%、70%、80%、90% 时，八项核心观测的准入集合不变，相对 70% 基准的 Kendall τ 、Spearman ρ 和 Top-5 重合率均为 1，Kimi K3 均列通用评价第 1。该结果只适用于当前覆盖结构。对非零任务-维度依赖 a_{stk} 作 $\pm 20\%$ 独立扰动并重新归一化，10,000 次试验中科研、日常和代码首名保持为 Kimi、Grok、Kimi 的比例分别为 0.9961、1.0000、1.0000，说明首名对本轮编码的小幅变化不敏感。

相对短板阈值取 $\theta = 0.20, 0.25, 0.30$ 时，科研资格数为 16、15、12，日常为 16、13、11，代码为 19、18、15。阈值改变工程候选集合，但不改变 S_{is} 的原始排序。由于 θ 表示当前候选集内的分位位置，模型样本变化后必须重算。

6.2 模型结构稳健性

Min-Max、Robust-Z 与 TOPSIS 对照 [24] 均保留 Kimi 的通用第 1；相对主模型的 Kendall τ 分别为 0.9358、0.9643 和 0.9415，Top-5 重合率均为 0.8。替代方法支持头部集合的稳定性，但没有证明具体名次不受评价尺度影响。

留一分析揭示了更有价值的结构敏感性。删除 AA-LCR 后，Claude Opus 5 升至通用第 1，Kimi 降至第 3；删除 C_4 长文本维度后，科研首名改为 Grok，Kimi 降至第 4；删除日常场景中的 C_3 知识边界校准维度后，GPT-5.6 Sol 升至首名，Kimi 降至第 3。其余留一 Benchmark、留一维度和模型家族结果见支撑材料。头部集合总体接近，但具体首名会依赖长文本和知识边界校准等关键维度。

6.3 工程决策稳健性

由于 L_s^{in}, L_s^{out} 与 τ_s 均属于情景参数，其具体取值具有决策者依赖性，第三问的稳健性重点考察工作负载和服务时延边界变化后，Pareto 集与预算推荐是否发生结构性改变。轻、中、重三档负载下，科研、日常和代码的质量-费用-时延三目标 Pareto 集 Jaccard 指数均为 1，前沿规模分别为 6、5、7；预算节点随 token 长度和单价变化，不能解释为固定金额。时延阈值从严格到宽松时，科研可行配置数保持 11，日常由 3 增至 8，代码由 9 增至 14，后二者的预算推荐路径随之改变。缓存命中率只改变费用和 Pareto 可行域，不改变 C_{ik} 或 S_{is} ；完整时延与缓存路径列于支撑材料。

成本-质量函数的留一模型检验中，科研单断点、日常对数和代码饱和形式分别有 5/11、9/10、9/14 次保持。科研和代码的具体函数形式受少数配置影响，因此本文只报告正向关联及横截面拟合，不把单断点或饱和形式解释为普适规律。

6.4 误差来源与适用边界

由于公开成绩可能受测试随机性、Prompt、Evaluation Harness、模型版本和网页更新影响；当前数据没有同一配置的重复复测，无法估计 Benchmark 测量方差。单项 Benchmark 也不能完整代表其对应评价维度。统一协议时延代理不能外推为本文三档工作负载下的场景响应时间。Dirichlet 排名概率和联合 Monte Carlo Pareto 频率只刻画本文预设权重、负载与时延空间内的不确定性，完整结果收入支撑材料，不能当作真实性能置信区间。而且多模态的能力由于数据不完善和部分模型该能力缺失，本模型并未将该能力纳入为评价因素。

本文结论限定于冻结日期的 27 个质量配置、20 个工程配置、三类任务链及既定价格和服务边界。速度、统一协议时延代理、价格和生成侧负担代理均未进入矩阵 C ；生成 token 数不等价于 kWh、GPU-hours 或完整推理 FLOPs。头部模型集合总体具有一定稳定性，但具体首名及工程推荐对关键评价维度和服务约束存在结构敏感性。模型版本、价格、评测协议、候选集合或业务任务改变后，应重新执行

$$X \rightarrow Z \rightarrow C \rightarrow S^G \rightarrow (S_{is}, I_{is}) \rightarrow D_s(B),$$

而不能沿用本文的分位位置和预算节点。

6.5 模型局限

除上述误差来源外，本文方法本身还有三处结构性局限，单列说明。

一是数据来源单一。全部质量观测取自 **Artificial Analysis** 一家的统一测评页面。这样做换来八项核心指标在同一评测协议下可比，是准入规则成立的前提，但评价结论也随之继承了该平台在题目选取、评测脚本和计分口径上的全部设定。换一套独立测评源，部分维度的相对排名可能移动。本文没有做多源交叉验证，所以所有排名只在“该平台口径下”成立，不宜读作模型能力的绝对次序。

二是能耗数据缺失，使问题三的工程侧不完整。候选配置里相当比例是闭源模型，厂商不公开参数量、推理硬件与运行功率，公开平台也没有同口径的 GPU-hours 或 kWh 观测。问题三的主模型于是只能用 token 直接费用和统一协议时延代理刻画落地成本，无法把真实算力消耗纳入 Pareto 前沿。要说明的是，token 单价只是厂商定价，不等同于单位能耗成本——它混合了硬件折旧、电力、利润与市场策略，两者相关却不成比例。本文的“成本”因此是采购口径的成本，对关注自建部署或能耗约束的使用者，现有结论只能提供部分依据。

三是候选面板的能力分布会影响相对得分的可读性。27 个质量配置里有若干在多数 Benchmark 上得分偏低的配置，它们带来两层影响。样本均值先被这部分配置压低，表 6 中 Kimi K3 各维度“相对均值”的正偏差随之被放大，读者不应把该列直接读成绝对领先幅度。更根本的是，分位数得分 $z_{ij} = (\text{rank}(x_{ij}) - 1) / (n_j - 1)$ 只记录排名位置、不记录能力差距：面板上多进若干弱配置，头部配置在每一维的分位数都会被动抬高，哪怕它的真实能力没变。换句话说，本文的综合得分衡量的是“在当前这批可比配置中的相对位置”，候选集合的强弱结构会直接影响得分数值。这也是 6.4 节强调候选集合变化后须重算分位位置的原因。

这三点不影响方法本身的有效性，但界定了结论的适用范围：单平台口径、采购侧成本、相对排名坐标。

7 结论与建议

7.1 问题一结论

在 27 个质量完整配置上，本文经 Benchmark 准入、正向化和分位数标准化，构造七项核心评价维度 C_1 至 C_7 。HLE 与 GPQA 融合为科学推理维度，其余维度采用单项透明映射。最大熵无偏好权重下，Kimi K3 (max) 以 0.824176 列第 1；其长文本、科学代码和交互任务维度突出，知识边界校准相对较弱。

7.2 问题二结论

任务链需求在固定矩阵 C 上形成差异化场景权重。科研长文本、日常对话、代码开发的场景综合得分首名依次为 Kimi K3 (0.777972)、Grok 4.6 (0.792308) 和 Kimi K3 (0.838612)。 $\theta = 0.25$ 只表示候选集内的相对短板约束，三类场景资格数依次为 15、13、18。日常场景更重交互与知识边界校准，代码场景更重科学代码与终端执行；该排名不等同于工程约束下的最终推荐。

7.3 问题三结论

在本文中等标准负载、冻结日价格和既定时延边界下，科研路径为 MiniMax-M3→Grok 4.6→Kimi K3，日常路径为 Muse Glimmer→Gemini 3.5 Flash-Lite→GLM-5.2，代码路径为 MiniMax-M3→GLM-5.2→Grok 4.6→Kimi K3。具体预算节点见表 13，它们不适用于其他任务长度、价格日期和服务边界。成本-质量拟合只作为横截面描述，科研和代码的具体函数形式对少数配置敏感。

7.4 使用建议

实际部署应同时读取场景、预算、时延、输入输出长度和价格日期；质量、时延和预算偏好对应不同可行域，不能以脱离场景的总排名替代工程决策。版本、协议、价格、任务规模或失败代价

变化后，应重新执行

$$X \rightarrow Z \rightarrow C \rightarrow S^G \rightarrow (S_{is}, I_{is}) \rightarrow D_s(B). \quad (20)$$

参考文献

[1] Artificial Analysis. Intelligence benchmarking[EB/OL]. 2026[2026-08-17]. <https://artificialanalysis.ai/methodology/intelligence-benchmarking>.

[2] PHAN L, GATTI A, HAN Z, et al. Humanity’s last exam[EB/OL]. arXiv, 2025[2026-08-17]. <https://arxiv.org/abs/2501.14249>.

[3] Artificial Analysis. Humanity’s last exam benchmark leaderboard[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/humanitys-last-exam>.

[4] REIN D, HOU B L, STICKLAND A C, et al. GPQA: a graduate-level google-proof Q&A benchmark[EB/OL]. arXiv, 2023 [2026-08-17]. <https://arxiv.org/abs/2311.12022>.

[5] Artificial Analysis. GPQA diamond benchmark leaderboard[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/gpqa-diamond>.

[6] J D, K W, C G, et al. AA-omniscience: evaluating cross-domain knowledge reliability in large language models[EB/OL]. arXiv, 2025[2026-08-17]. <https://arxiv.org/abs/2511.13029>.

[7] Artificial Analysis. AA-omniscience: knowledge and hallucination benchmark[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/omniscience>.

[8] Artificial Analysis. Announcing artificial analysis long context reasoning (AA-LCR)[EB/OL]. 2025[2026-08-17]. <https://artificialanalysis.ai/articles/announcing-aa-lcr>.

[9] Artificial Analysis. Artificial analysis long context reasoning benchmark leaderboard[EB/OL]. Artificial Analysis, 2026 [2026-08-17]. <https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning>.

[10] TIAN M, GAO L, ZHANG S D, et al. SciCode: a research coding benchmark curated by scientists[EB/OL]. arXiv, 2024 [2026-08-17]. <https://arxiv.org/abs/2407.13168>.

[11] Artificial Analysis. SciCode benchmark leaderboard[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/scicode>.

[12] M A M, A G S, C N, et al. Terminal-bench: benchmarking agents on hard, realistic tasks in command line interfaces[EB/OL]. arXiv, 2026[2026-08-17]. <https://arxiv.org/abs/2601.11868>.

[13] Terminal-Bench Team. Terminal-bench 2.1[EB/OL]. 2026[2026-08-17]. <https://www.tbench.ai/news/terminal-bench-2-1>.

[14] Artificial Analysis. Terminal-bench v2.1 benchmark leaderboard[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/terminalbench-v2-1>.

[15] YAO S, SHINN N, RAZAVI P, et al. tau-bench: a benchmark for tool-agent-user interaction in real-world domains[EB/OL]. arXiv, 2024[2026-08-17]. <https://arxiv.org/abs/2406.12045>.

[16] Artificial Analysis. tau3-banking benchmark leaderboard[EB/OL]. Artificial Analysis, 2026[2026-08-17]. <https://artificialanalysis.ai/evaluations/tau3-banking>.

[17] SPEARMAN C. The proof and measurement of association between two things[J/OL]. The American Journal of Psychology, 1904, 15(1): 72-101[2026-08-17]. <https://www.jstor.org/stable/1412159>. DOI: 10.2307/1412159.

[18] SciPy Developers. scipy.stats.spearmanr[EB/OL]. SciPy Reference Guide, 2026[2026-08-17]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html>.

[19] SciPy Developers. scipy.cluster.hierarchy.linkage[EB/OL]. SciPy Reference Guide, 2026[2026-08-17]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html>.

[20] JAYNES E T. Information theory and statistical mechanics[J/OL]. Physical Review, 1957, 106(4): 620-630[2026-08-17]. <https://link.aps.org/doi/10.1103/PhysRev.106.620>.

[21] Moonshot AI. Kimi K3: open frontier intelligence[EB/OL]. 2026[2026-08-17]. <https://www.kimi.com/blog/kimi-k3>.

[22] EMMERICH M T M, DEUTZ A H. A tutorial on multiobjective optimization: fundamentals and evolutionary methods[J/OL]. Natural Computing, 2018, 17: 585-609[2026-08-17]. <https://link.springer.com/article/10.1007/s11047-018-9685-y>.

[23] National Institute of Standards and Technology. How can I tell if a model fits my data?[EB/OL]. NIST/SEMATECH e-Handbook of Statistical Methods, 2026[2026-08-17]. <https://www.itl.nist.gov/div898/handbook/pmd/section4/pmd44.htm>.

[24] HWANG C L, YOON K. Multiple attribute decision making: methods and applications[M/OL]. Berlin: Springer, 1981 [2026-08-17]. <https://link.springer.com/book/10.1007/978-3-642-48318-9>.

A 支撑材料与复现清单

支撑材料文件命名为 **TMML2026010.zip**，包含原始网页记录与截图、冻结数据、来源和哈希清单、完整程序、参数、结果表和图表。正文关键数字均可由支撑材料入口脚本复算。支撑材料不包含 API Key、聊天记录或浏览器缓存。

表 15 支撑材料目录及复现用途

路径	内容与复现用途
data/ evidence/ tables/ figures/ paper/ code/	公开网页 JSON、正文快照和访问时间、冻结观测 Benchmark 榜单、价格和工程字段截图 完整排名、权重、稳健性与成本拟合结果 流程、覆盖、相关、排名、场景、预算和成本拟合图 pdf 的 latex 的源码及配套的图片 存放整合过的全部代码

其中 `tables/all_high_corr_pairs.csv` 枚举全部 $\rho \geq 0.85$ 的指标对，并记录构念、冗余判定、处理方式和

理由;ability_scores.csv、task_chain_demands.csv、scenario_weights.csv、eligibility_status.csv、q3_pareto_quality_cost_latency_*.csv 与 q3_budget_segments_*.csv 分别对应 C 、任务需求、场景权重、相对短板资格、三目标 Pareto 和预算选型。能耗扩展模块（5.6 节）所用的硬件参数、电价、PUE 等全部数据及其官方出处、访问日期与完整计算过程，汇总于 能耗支撑数据.pdf。

B AI 工具使用说明

本文的建模思路、模型设计、公式推导、参数选取与结果分析均由参赛队员独立完成。生成式人工智能仅作为辅助工具，参与以下环节：一是辅助查找文中表述不严谨、前后不一致之处，并对行文措辞与格式作局部润色，最终表述以队员复核为准；二是辅助检索公开资料并定位数据来源，资料的真实性、适用性与最终采用由队员逐一核验；三是辅助编写部分数据整理与绘图脚本，脚本的功能设计、运行与结果核对均由队员完成；四是参与方案讨论，提供参考性思路，模型的确定与取舍由队员自主决定。图 1 由图像生成工具辅助美化，其流程语义、文字与箭头关系均由参赛队员核验。参赛队员对全部数据来源、模型假设、参数取值、计算结果、图表与最终文稿进行人工复核，并对论文结论负全部责任。生成式人工智能未作为测评数据来源，未参与模型成绩的获取与整理，未生成或改写任何模型成绩。以上为实际使用范围；统一表格另填。

C 核心代码节选

由于代码较长，将完整代码放在支撑材料里，以下展示节选主程序 analyze.py 中与正文模型直接对应的核心函数，按正文顺序排列：分位数标准化与七维合成（rank_percentile、analyze_scores、normalize_weights）、跨场景贡献分解（scenario_decomposition）、Pareto 前沿与预算分段（pareto_frontier、budget_segments）、成本-质量拟合（fit_models）。受篇幅所限仅列主体逻辑，完整代码（含数据准入、覆盖率筛选、稳健性分析与全部绘图）见支撑材料。

Listing 1 核心代码节选（analyze.py）

```
def rank_percentile(values: Sequence[float], higher_is_better: bool = True) -> np.ndarray:
    arr = np.asarray(values, dtype=float)
    ranks = rankdata(arr if higher_is_better else -arr, method="average")
    if len(arr) <= 1:
        return np.full(len(arr), 0.5)
    return (ranks - 1.0) / (len(arr) - 1.0)

def minmax(values: np.ndarray) -> np.ndarray:
    arr = np.asarray(values, dtype=float)
    lo = float(np.min(arr))
    hi = float(np.max(arr))
    if hi - lo <= 1e-12:
        return np.full(arr.shape, 0.5)
    return (arr - lo) / (hi - lo)

def analyze_scores(raw: np.ndarray, names: Sequence[str], normalization: str = "quantile") -> dict[str, object]:
    oriented = np.asarray(raw, dtype=float)
    if normalization == "quantile":
        z = np.column_stack([rank_percentile(oriented[:, j], higher_is_better=True) for j in range(oriented.shape[1])])
    elif normalization == "minmax":
        z = np.column_stack([minmax(oriented[:, j]) for j in range(oriented.shape[1])])
    elif normalization == "robust-z":
        z = np.column_stack([robust_z(oriented[:, j]) for j in range(oriented.shape[1])])
    else:
        raise ValueError(normalization)
    # Transparent mapping: CI is the equal average of HLE and GPQA; every
    # other quality dimension is a single observed benchmark. Speed and EZE
    # are deliberately absent from the quality matrix.
    q_by_ability = {
        ability: np.full(sum(item["ability"] == ability for item in INDICATORS), 1.0 / sum(item["ability"] == ability for item in INDICATORS))
        for ability in ABILITIES
    }
    ability_scores = np.column_stack([
        np.sum(z[:, [j for j, item in enumerate(INDICATORS) if item["ability"] == ability]] * q_by_ability[ability][None, :], axis=1)
        for ability in ABILITIES
    ])
    generic_weights = np.full(len(ABILITIES), 1.0 / len(ABILITIES))
    scenario_weights = {scenario: normalize_weights([SCENARIO_DEMAND[scenario][ability] for ability in ABILITIES]) for scenario in SCENARIOS}
    all_weights = {"问题-通用": generic_weights, **scenario_weights}
    score_tables = {key: ability_scores @ np.asarray(weights, dtype=float) for key, weights in all_weights.items()}
    rank_tables = {key: rankdata(-score, method="average") for key, score in score_tables.items()}
    eligibility = {
        scenario: {theta: np.all(ability_scores[:, [ABILITIES.index(a) for a in CRITICAL_ABILITIES[scenario]]] >= theta, axis=1) for theta in (0.20, 0.25, 0.30)}
        for scenario in SCENARIOS
    }
    return {
        "oriented": oriented,
        "z": z,
        "q_by_ability": q_by_ability,
        "ability_scores": ability_scores,
        "weights": all_weights,
        "scores": score_tables,
        "ranks": rank_tables,
        "eligibility": eligibility,
        "normalization": normalization,
        "names": list(names),
    }
```

```

def normalize_weights(values: Sequence[float]) -> np.ndarray:
    arr = np.asarray(values, dtype=float)
    total = float(np.sum(arr))
    return arr / total if total else np.full(len(arr), 1.0 / len(arr))

def rank_rows(names: Sequence[str], scores: Mapping[str, np.ndarray], ranks: Mapping[str, np.ndarray]) -> list[dict[str, object]]:
    rows = []
    for i, name in enumerate(names):
        row: dict[str, object] = {"model_name": name}
        for key, values in scores.items():
            row[f"{key}_score"] = round(float(values[i]), 8)
            row[f"{key}_rank"] = int(round(float(ranks[key][i])))
        rows.append(row)
    return rows

def pareto_frontier(rows: Sequence[Mapping[str, object]], performance_key: str, cost_key: str) -> list[dict[str, object]]:
    sorted_rows = sorted(rows, key=lambda row: (float(row[cost_key]), -float(row[performance_key])))
    best = -float("inf")
    frontier = []
    for row in sorted_rows:
        performance = float(row[performance_key])
        if performance > best + 1e-12:
            frontier.append(dict(row))
            best = performance
    return frontier

def budget_segments(rows: Sequence[Mapping[str, object]], score_key: str, cost_key: str, latency_key: str, latency_limit: float) -> list[dict[str, object]]:
    eligible = [row for row in rows if float(row[latency_key]) <= latency_limit]
    if not eligible:
        return [{"budget_lower_usd_per_1000": 0.0, "budget_upper_usd_per_1000": "inf", "recommended_model": "无可行模型", "score": "", "latency_seconds": "", "latency_limit": latency_limit, "status": "无可行模型"}]
    min_cost = min(float(row[cost_key]) for row in eligible)
    unique_costs = sorted((round(float(row[cost_key]), 10) for row in eligible))
    switches: list[tuple[float, Mapping[str, object]]] = []
    previous_model = None
    for cost in unique_costs:
        candidates = [row for row in eligible if float(row[cost_key]) <= cost + 1e-9]
        # At a tied cost, keep the highest-performance model. This prevents
        # zero-width intervals such as [64,64] from entering the table.
        best_row = max(candidates, key=lambda row: (float(row[score_key]), -float(row[latency_key])))
        if best_row["model_name"] != previous_model:
            switches.append((cost, best_row))
            previous_model = best_row["model_name"]
    segments = []
    if min_cost > 1e-12:
        segments.append({"budget_lower_usd_per_1000": 0.0, "budget_upper_usd_per_1000": round(min_cost, 8), "recommended_model": "无可行模型", "score": "", "latency_seconds": "", "latency_limit": latency_limit, "status": "无可行模型"})
    for i, (start, row) in enumerate(switches):
        end = switches[i + 1][0] if i + 1 < len(switches) else float("inf")
        if end <= start + 1e-12:
            continue
        segments.append({"budget_lower_usd_per_1000": round(start, 8), "budget_upper_usd_per_1000": "inf" if math.isinf(end) else round(end, 8), "recommended_model": row["model_name"], "score": round(float(row[score_key]), 8), "latency_seconds": round(float(row[latency_key]), 8), "latency_limit": latency_limit})
    return segments

TOKEN_CHART_FILES = {
    "HLE": "hle-charts.json",
    "GPQA": "gpqa-diamond-charts.json",
    "LCR": "aa-lcr-charts.json",
    "SciCode": "scicode-charts.json",
    "tau3": "tau3-banking-charts.json",
    "Terminal": "terminalbench-v2-1-charts.json",
}

def parse_token_label(label: object) -> float | None:
    """Parse the compact labels used by the public Token Usage SVGs."""
    text = str(label).strip().replace(",", "")
    if not text:
        return None
    match = re.fullmatch(r"([0-9]+(?:\.[0-9]+)?)\s*([kK])?", text)
    if not match:
        return None
    number = float(match.group(1))
    if match.group(2):
        number *= 1000.0
    return number

def load_compute_proxy(names: Sequence[str]) -> dict[str, dict[str, object]]:
    return {}

def scenario_decomposition(names: Sequence[str], analysis: Mapping[str, object]) -> None:
    records = []
    scenarios = list(SCENARIO_INTENSITY)
    for a_i in range(len(scenarios)):
        for b_i in range(a_i + 1, len(scenarios)):
            a, b = scenarios[a_i], scenarios[b_i]
            delta_w = np.asarray(analysis["weights"][b]) - np.asarray(analysis["weights"][a])
            for i, name in enumerate(names):
                row = {"model_name": name, "from_scenario": a, "to_scenario": b}
                contributions = []
                for j, ability in enumerate(ABILITIES):
                    value = float(delta_w[j] * analysis["ability_scores"][i, j])
                    row[f"delta_{ability}"] = round(value, 8)
                    contributions.append(value)
                row["delta_total"] = round(float(np.sum(contributions)), 8)
                row["direct_score_difference"] = round(float(analysis["scores"][b][i] - analysis["scores"][a][i]), 8)
                row["rank_from"] = int(analysis["ranks"][a][i])
                row["rank_to"] = int(analysis["ranks"][b][i])

```

```

        records.append(row)
    write_csv(RESULTS / "scenario_decomposition.csv", records)

def fit_models(x: np.ndarray, y: np.ndarray) -> dict[str, dict[str, object]]:
    """Fit the five prespecified curves and evaluate LOOCV RMSE/AICC."""
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    eps = max(float(np.min(x)) * 1e-6, 1e-8)

    def linear_fit(xx: np.ndarray, yy: np.ndarray) -> tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]:
        coef = np.linalg.lstsq(np.column_stack([np.ones(len(xx)), xx]), yy, rcond=None)[0]
        return lambda v: coef[0] + coef[1] * np.asarray(v), 2, {"a": float(coef[0]), "b": float(coef[1])}

    def log_fit(xx: np.ndarray, yy: np.ndarray) -> tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]:
        coef = np.linalg.lstsq(np.column_stack([np.ones(len(xx)), np.log(xx + eps)]), yy, rcond=None)[0]
        return lambda v: coef[0] + coef[1] * np.log(np.asarray(v) + eps), 2, {"a": float(coef[0]), "b": float(coef[1]), "eps": eps}

    def power_fit(xx: np.ndarray, yy: np.ndarray) -> tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]:
        safe_y = np.maximum(yy, 1e-8)
        coef = np.linalg.lstsq(np.column_stack([np.ones(len(xx)), np.log(xx + eps)]), np.log(safe_y), rcond=None)[0]
        a, b = float(np.exp(coef[0])), float(coef[1])
        return lambda v: a * np.power(np.asarray(v) + eps, b), 2, {"a": a, "b": b, "eps": eps}

    def sat_fn(v: np.ndarray, a: float, b: float, tau: float) -> np.ndarray:
        return a - b * np.exp(-np.asarray(v) / max(tau, 1e-8))

    def saturation_fit(xx: np.ndarray, yy: np.ndarray) -> tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]:
        p0 = [float(np.max(yy)), float(np.max(yy) - np.min(yy)), max(float(np.median(xx)), 1e-4)]
        bound_lo = [-2.0, -2.0, 1e-8]
        bound_hi = [2.0, 4.0, max(float(np.max(xx) * 100.0), 1.0)]
        params, _ = curve_fit(sat_fn, xx, yy, p0=p0, bounds=(bound_lo, bound_hi), maxfev=20_000)
        return lambda v: sat_fn(np.asarray(v), *params), 3, {"a": float(params[0]), "b": float(params[1]), "tau": float(params[2])}

    def breakpoint_fit(xx: np.ndarray, yy: np.ndarray) -> tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]:
        candidates = np.unique(np.quantile(xx, np.linspace(0.15, 0.85, 15)))
        best = None
        for knot in candidates:
            design = np.column_stack([np.ones(len(xx)), xx, np.maximum(0.0, xx - knot)])
            coef = np.linalg.lstsq(design, yy, rcond=None)[0]
            resid = yy - design @ coef
            rss = float(np.sum(resid * resid))
            if best is None or rss < best[0]:
                best = (rss, float(knot), coef)
        if best is None:
            raise ValueError("断点候选为空")
        _, knot, coef = best
        return lambda v: np.column_stack([np.ones(len(np.asarray(v))), np.asarray(v), np.maximum(0.0, np.asarray(v) - knot)]) @ coef, 4, {"a": float(coef[0]), "b1": float(coef[1]), "b2_change": float(coef[2]), "knot": knot}

    fitters: dict[str, Callable[[np.ndarray, np.ndarray], tuple[Callable[[np.ndarray], np.ndarray], int, dict[str, float]]]] = {
        "线性": linear_fit, "对数": log_fit, "幂函数": power_fit, "饱和": saturation_fit, "单断点": breakpoint_fit,
    }
    outputs: dict[str, dict[str, object]] = {}
    n = len(x)
    for name, fitter in fitters.items():
        try:
            pred_fn, p, params = fitter(x, y)
            pred = np.asarray(pred_fn(x), dtype=float)
            residual = y - pred
            rss = float(np.sum(residual * residual))
            aicc = n * math.log(max(rss / n, 1e-12)) + 2 * p + (2 * p * (p + 1) / max(n - p - 1, 1))
            loocv_errors = []
            for holdout in range(n):
                mask = np.ones(n, dtype=bool)
                mask[holdout] = False
                try:
                    fold_fn, _, _ = fitter(x[mask], y[mask])
                    loocv_errors.append(float(y[holdout] - fold_fn(np.asarray([x[holdout]])))[0])
                except Exception:
                    loocv_errors.append(float("nan"))
            valid = np.asarray(loocv_errors, dtype=float)
            cv_rmse = float(np.sqrt(np.nanmean(valid * valid))) if np.any(np.isfinite(valid)) else float("inf")
            outputs[name] = {"params": params, "n_params": p, "pred": pred, "residual": residual, "rss": rss, "aicc": float(aicc), "loocv_rmse": cv_rmse}
        except Exception as exc:
            outputs[name] = {"params": {}, "n_params": 0, "pred": np.full(n, np.nan), "residual": np.full(n, np.nan), "rss": np.nan, "aicc": np.inf, "loocv_rmse": np.inf, "error": str(exc)}
    return outputs

def cost_fit_outputs(scenario_rows: Mapping[str, Sequence[Mapping[str, object]]]) -> None:
    summary = []
    point_rows = []
    for scenario, rows in scenario_rows.items():
        # Cost-performance curves are an engineering extension of the hard
        # gate, so non-recommendable quality configurations do not determine
        # the selected curve or its budget interpretation.
        rows = [row for row in rows if row.get("eligible_theta_025") in {True, "True"}]
        if len(rows) < 3:
            raise ValueError(f"{scenario} I_is(0.25) 相对短板资格样本不足以拟合成本曲线: {len(rows)}")
        x = np.asarray([float(row["cost_usd_per_1000_tasks"]) for row in rows])
        y = np.asarray([float(row["performance"]) for row in rows])
        fits = fit_models(x, y)
        best = min(fits, key=lambda name: (float(fits[name]["loocv_rmse"]), float(fits[name]["aicc"])))
        rank_corr, rank_p = spearmanr(x, y)
        for model, result in fits.items():
            tss = float(np.sum((y - np.mean(y)) ** 2))
            r2 = float(1.0 - result["rss"] / tss) if tss > 1e-12 and np.isfinite(result["rss"]) else float("nan")

```